

A DATA-DRIVEN COMPARISON OF HOW JUNIOR AND SENIOR DEVELOPERS USE NATURAL LANGUAGE TO GENERATE REACT COMPONENTS

Athira Allaveedu Ashok

Masters in Computer Science: Software Engineering

Supervisor: Associate Professor, Dr. sc. ing. Samuel Moveh

INTRODUCTION

Background: LLMS such as ChatGPT enable developers to generate source code from natural language prompt, making AI-assisted development prevalent. Understanding how developer experience affects this process is critical for improving software quality.

Research Aim: To investigate how developer experience influences natural language prompt formulation for generating React components, and how prompt differences affect AI-generated code quality.

Hypothesis: Experienced developers use more structured and technically specific prompts, resulting in higher quality AI-generated React components (lower complexity, higher maintainability, fewer errors).

Research Questions: RQ1: How do junior and senior developers differ in prompt formulation, and what is the effect on code quality? RQ2: Can clustering techniques reveal experience-based prompting patterns? RQ3: Which prompt and code features most distinguish the two groups?

Research Gap: Most prior studies used predefined prompts. Few examined how developer experience naturally shapes prompt writing and its downstream effect on code quality in front-end frameworks like React.

METHODOLOGY

•150 developers (75 junior: <3 yrs, 75 senior: ≥5 yrs) each submitted one natural language prompt for a React To-Do app via Google Forms

•All 150 prompts were individually entered into ChatGPT; generated React code was run in VS Code using Vite

•Prompts scored (1–5) on Clarity, Specificity, and Structure; code analysed for ESLint errors, Cyclomatic Complexity, Maintainability Index, and Readability Score

•Statistical analysis: Welch's t-test, Cohen's d, Pearson correlation, and K-means clustering (k=2) in Python

KEY REFERENCES

[1] White, J., Hays, S., Fu, Q., Spencer-Smith, J. and Schmidt, D.C. (2023) ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design, *arXiv Preprint*.

[2] Khojah, R., Gomes de Oliveira Neto, F., Mohamad, M. and Leitner, P. (2025) The impact of prompt programming on function-level code generation, *IEEE Transactions on Software Engineering*. <https://doi.org/10.1109/TSE.2025.3587794>

ACKNOWLEDGEMENT

The author sincerely thanks all 150 software developers who voluntarily contributed their prompts.

CONTACTS

Athira Allaveedu Ashok

Email id: athiraathu1497@gmail.com

GitHub: github.com/Athira-A-Ashok/MasterThesisExperiment



RESULTS AND DISCUSSION

Prompt Quality ($p < 0.001$): Senior developers produced significantly longer (676 vs 202 chars), clearer (4.99 vs 3.52), more specific (4.67 vs 2.53), and better-structured (4.31 vs 2.15) prompts. All three quality dimensions had very large effect sizes ($d = -2.83, -3.21, -3.74$).

Maintainability Index ($p < 0.001, d = -1.286$): Senior-prompted code averaged MI = 48.80 (moderately maintainable) vs junior-prompted MI = 37.58 (below threshold of 40). This 11.22-point gap has strong practical significance for long-term code quality.

Other Code Quality Metrics: Junior code had higher Cyclomatic Complexity (4.70 vs 3.87, $p = 0.036$), more ESLint errors (17.3% vs 5.3%, $p = 0.021$), and worse readability score (7.38 vs 6.78, $p = 0.041$).

Correlation Analysis: Prompt specificity ($r = 0.413, p < 0.001$) and structural organisation ($r = 0.379, p < 0.001$) positively correlated with MI across all 150 participants. Prompt length alone was weakly correlated ($r = 0.222$), confirming that technical content and structure — not sole length — drive code quality.

K-Means Clustering ($k=2$, Silhouette=0.48): Cluster 0 (82 members: 72 junior, 10 senior) — shorter prompts, lower prompt quality, MI = 36. Cluster 1 (68 members: 65 senior, 3 junior) — longer structured prompts, MI = 52. Crossover cases confirm prompt engineering is a learnable skill.

Limitations: Single standardised task; single-rater prompt scoring without inter-rater reliability testing; individual prior AI tool experience not controlled.

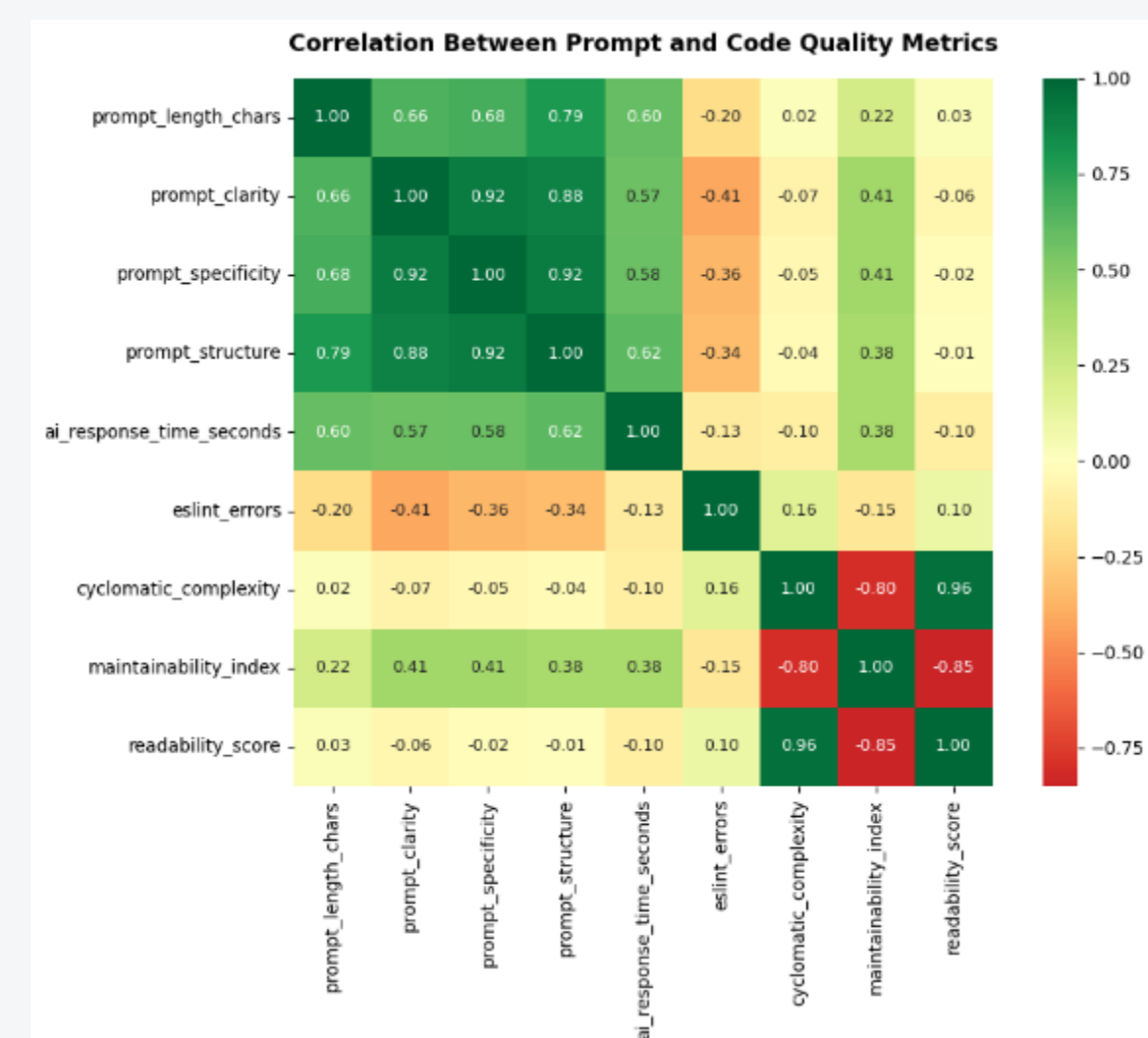


Figure 1. Correlation Heatmap

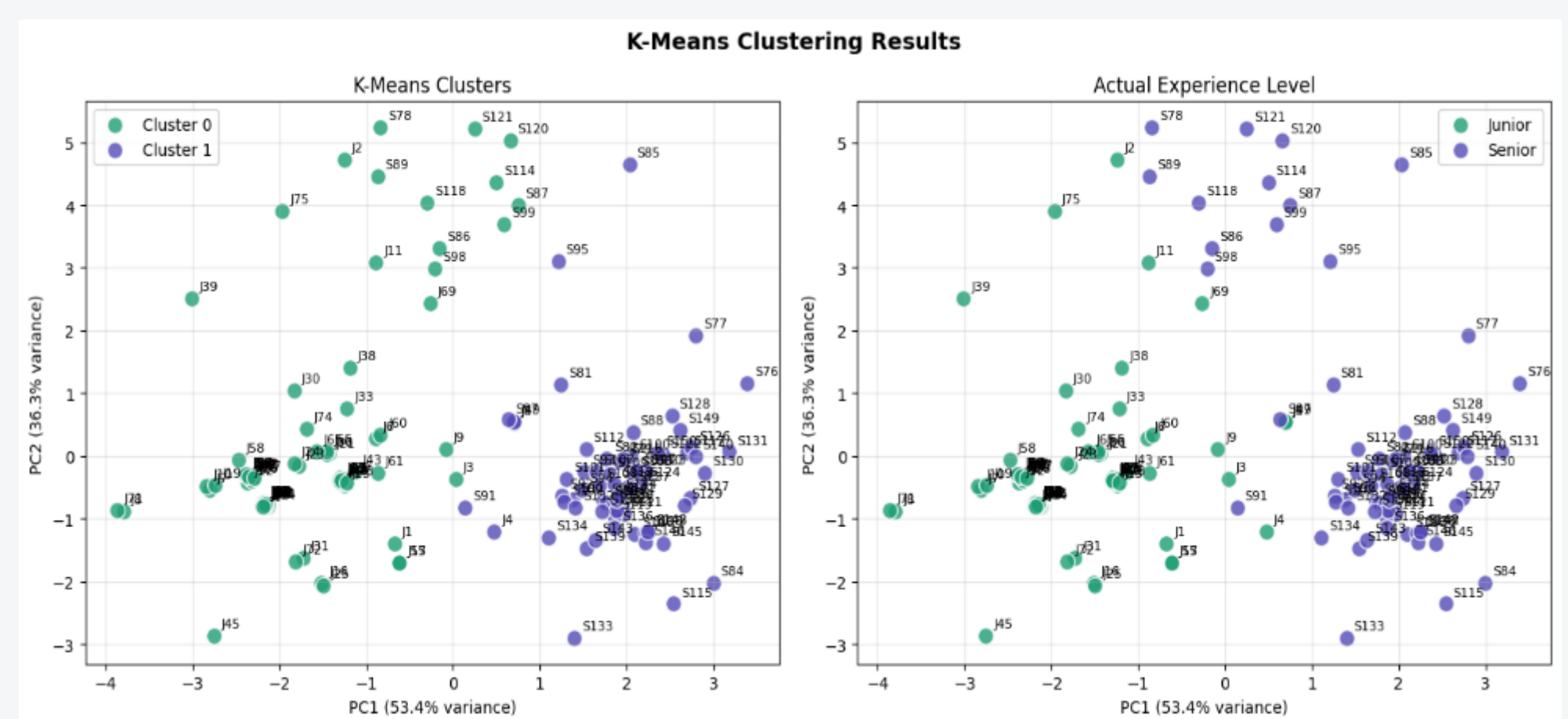


Figure 2. K-Means PCA Scatter Plot

CONCLUSIONS

Senior developers produced significantly clearer, more specific, and better-structured prompts ($p < 0.001, d > 2.8$), resulting in higher-quality AI-generated code (MI = 48.80 vs 37.58). Prompt specificity and structure are the strongest predictors of code maintainability. Clustering confirmed that structured prompting is a learnable skill, independent of experience level, highlighting the need for formal prompt engineering training for all developers.